CPU power limits: experimental design

INF-2204 - 2

Group and project

Payam Nami, pna020@uit.no
William-Nima Felahi, wfe002@uit.no

We will use the desktop from Task 1 to test what happens when the CPU is allowed less power. The questions are still about performance, differences between workloads, energy efficiency and temperature. This report describes how we plan to measure them. No CPU benchmark results have been collected for this project yet.

Our expectation is conditional: a higher limit should help only when the workload actually reaches the power limit. Otherwise, changing the BIOS number may do very little. We want to check that rather than assume every program behaves like a full-core render.

1. Hardware environment

CPU	Current: delidded Ryzen 7 9850X3D with Conductonaut Extreme. Planned for the experiments: delidded AMD Ryzen 9 9950X, Zen 5, 16 cores / 32 threads, 64 MB L3, 170 W default TDP and 95 C maximum operating temperature [1].
Motherboard	ASRock X870E Taichi OCF, AM5. Current BIOS: 4.41.
Memory	32 GB V-Color DDR5, 2 x 16 GB, TMXFL1680838KWK. The current 8000 MT/s profile is stable on the 9850X3D. The sticks are currently air-cooled.
GPU	NVIDIA GeForce RTX 5080. Same driver and default GPU configuration throughout; no GPU tuning.
Storage	Samsung 9100 PRO 2 TB and Lexar NM790 2 TB. Planned input/output location: the Samsung SSD, with enough free space for all runs.
Current cooling	OCOCOO H09C water-cooling pump, Iceman AM5 waterblock and Alphacool GL50 coolant. Two 360 mm radiators, one 40 mm thick and the other 36 mm, both with push-pull fans. Loop: pump -> bottom radiator -> GPU -> CPU -> top radiator -> reservoir.

The CPU and cooling changes are planned for early next week (12-18 October). We will add waterblocks for the RAM and an Alphacool UT60 420 mm radiator with push-pull fans at the bottom, keeping both 360 mm radiators. The new loop will be: pump -> bottom 420 mm radiator -> GPU -> CPU -> top radiator -> side radiator -> RAM -> reservoir.

The current memory settings are saved in ZenTimings: 8000 MT/s, MCLK 4000 MHz, UCLK 2000 MHz and FCLK 2200 MHz. We will save the full timings too. After the CPU swap, we will check the same profile on the 9950X before keeping it fixed across all PPT settings. All main measurements will use the finished 9950X setup.

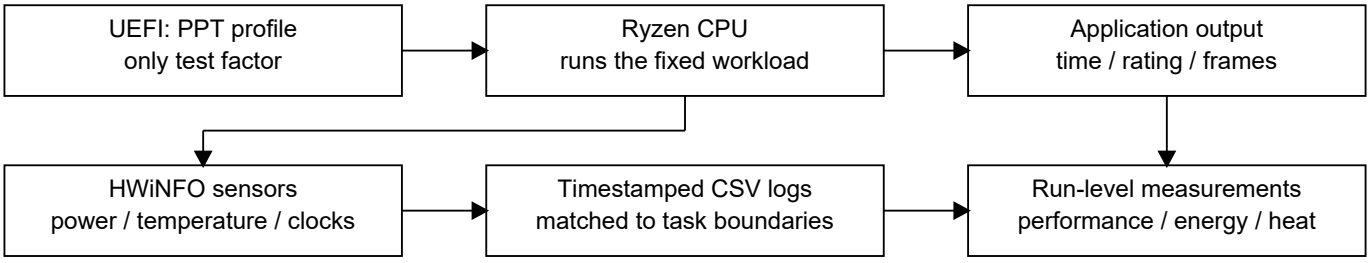


Figure 1. Planned measurement path on one PC. PPT changes the CPU's power allowance; application outputs and sensor logs are joined using the same run boundaries. Arrows show data or control flow, not measured results. Diagram by the group.

2. Software environment

The OS will be Windows 11 Pro. We plan to use Windows Balanced for the whole campaign and keep the same chipset and GPU drivers, security settings and hardware-accelerated GPU scheduling setting. We will save their configuration before testing. Updates, downloads, overlays and other monitoring programs must not run during a recorded test.

Tools	HWiNFO64 8.34 and 7-Zip 25.01 x64 are currently available. Planned additions: Blender 4.5 LTS, HandBrakeCLI, 3DMark and PresentMon Console. Confirm access to both 3DMark benchmarks, then record and freeze tool and test versions after the pilot.
Logging	HWiNFO CSV logging at a planned 1000 ms interval [3]. Log CPU package power, CPU (Tctl/Tdie) temperature, effective clocks, PPT/TDC/EDC usage and limit flags. Also log GPU use during 3DMark.
Frame capture	If the pilot confirms usable capture, PresentMon targets the active 3DMark graphics-test process and main swap chain. Use legacy CSV mode for MsBetweenPresents and absolute timestamps; keep the capture settings fixed [9].

No simulator or custom benchmark code is needed for this design. We will use the tools' existing commands and saved presets. The final commands, presets and versions will be kept with the data; analysis can be done in a spreadsheet using the formulas in Section 5.

3. Workloads

CPU rendering

Blender will render one fixed Classroom frame with Cycles on the CPU only, using 32 threads on the 9950X [4, 5]. The planned settings are frame 1, 1920 x 1080, 128 samples, fixed seed, adaptive sampling off and denoising off. We will save the configured scene. This refines Task 1's Blender test into a fixed render, so each run finishes the same work rather than relying on a time-limited score.

Compression and video encoding

For 7-Zip we plan `7z b 10 -mmt32 -md25`: ten internal iterations, 32 requested benchmark threads and an upper dictionary size of 2^25 bytes (32 MiB). The command can test several dictionary sizes; we will retain the complete output and compare the same final average compression and decompression ratings separately [6]. These are benchmark MIPS ratings, not hardware instruction counters.

HandBrake will encode the first 180 seconds of one fixed Tears of Steel source. We plan software x265, 8-bit output, medium encoder preset, RF 22 and constant 24 FPS. Keep the source dimensions, disable cropping, filters, audio, subtitles and hardware decoding, and write an MKV file. This is a sustained CPU workload; NVENC or another GPU encoder would answer a different question [7, 8].

3DMark CPU and graphics benchmarks

We plan to use 3DMark instead of Cyberpunk's benchmark. Its fixed tests should be easier for us to repeat. Time Spy at its default 2560 x 1440 setting has two graphics tests and a CPU test [10]; we will keep the CPU and graphics results separate. CPU Profile tests 1, 2, 4, 8, 16 and maximum threads, with little GPU load [15]. We will run all six and report each score. These give us different CPU demands without adding more games. Keep defaults, disable demos where possible and freeze the settings in the pilot. None of these tests represents every game.

4. Datasets and models

The inputs are the official Classroom scene (CC0) [5], the 4K Tears of Steel source described in HandBrake's documentation (3840 x 1714, 24 FPS) [8], 7-Zip's internally generated LZMA test data [6], and 3DMark's built-in scenes and CPU workloads. The scene and video provide repeatable real application inputs; 7-Zip needs no external archive. Save source URLs, licences, file sizes and SHA-256 hashes before testing, plus the edited scene and encode preset. We will not build a separate simulation model.

5. Evaluation and performance metrics

Each measurement belongs to one completed run at one PPT profile. The stock profile is the comparison point. We will keep the configured PPT in watts separate from measured package power: the cap is a ceiling, not the power the CPU necessarily uses.

Performance and timing

For Blender and HandBrake, task time T is wall-clock seconds from process launch to successful exit, including loading and writing. Lower is better; for percentage comparisons use S = 1/T. For 7-Zip, S is its compression or decompression rating. For 3DMark, use the reported CPU, graphics or thread-count score, compared only with the same test at stock. Save reported FPS too; do not use the combined Time Spy score alone.

Where capture works, use MsBetweenPresents for each graphics test separately [9], excluding loading and transitions. Let N be the captured interval count and T their total duration. Match power samples to these timestamps. Average FPS is 1000 divided by mean frame time in milliseconds; also report median, P95 and P99 frame time. Our 1% low FPS is 1000 divided by the mean of the slowest ceil(0.01 x N) intervals. These describe benchmark frame pacing, not display latency. CPU Profile uses its own scores, not these frame statistics.

Package energy, power and efficiency

HWiNFO provides the power samples; it is not an external electricity meter [3]. We will match its timestamps to the task interval, interpolate only the two boundary values and use the actual spacing between samples:

$$E_{pkg} \text{ (J) approx. sum of } [(P_i + P_{i+1})/2] \times (t_{i+1} - t_i)$$
$$\text{Mean package power (W)} = E_{pkg} / T$$

P is logged CPU package power in watts and t is time in seconds. We will not calculate energy as PPT x time. The main value includes the package's idle/background contribution during the task, without subtracting an idle estimate. It excludes GPU, PSU and wall-socket energy. Software energy measurement has limitations [11]; the confidence intervals below will not tell us the sensor's absolute accuracy.

For renders and encodes, report joules per job; performance per watt is $(1/T)/(E_{pkg}/T) = 1/E_{pkg}$. For 7-Zip, divide each rating by mean package watts over the full benchmark; equal iteration counts may not mean equal bytes processed. For 3DMark graphics tests, use FPS/W and E_{pkg}/N in J/frame for matched captures. Report CPU-test scores and package energy separately. Integrate each named active test, excluding loading; if a short test cannot be aligned reliably, report its score without an energy estimate.

Comparisons, heat and uncertainty

Within each workload, normalised performance is $100 \times S/S_{stock}$. Between adjacent numeric PPT limits, report performance change as $100 \times (S_{next}/S_{current} - 1)$. For fixed jobs and matched CPU tests, report energy change the same way using E. For graphics tests, use J/frame; for 7-Zip, compare rating/W. A gain-per-extra-energy ratio is only useful when the energy increase is positive. If it is zero or negative, show the two changes instead of forcing a ratio.

Report mean and peak CPU temperature in C, average effective clock and the proportion of samples with PPT usage at least 95%. Check TDC, EDC and thermal flags too: a flat performance curve could be another limit, not proof that PPT never matters [2]. For longer renders and encodes, also report mean temperature over the final 60 seconds when the job lasts at least three minutes.

Use 12 valid, separate runs per condition. Show raw run points, mean, sample standard deviation and approximate 95% confidence intervals: mean +/- $t_{0.975,n-1}$ x s/sqrt(n), with n = 12 [12]. The calculation assumes independent runs and an approximately normal spread of values [13, 14]. Twelve runs help us estimate variation, but do not guarantee a precise result. We will check the raw points for skew and drift over time. Frames and one-second sensor samples are not extra repetitions. Planned plots put PPT (W) on the horizontal axis and performance (% of stock), energy (J/job or J/frame), efficiency or temperature (C) on labelled vertical axes. Stock is a separately labelled point until its actual PPT is known.

6. Experimental protocols

Power profiles and controlled settings

The main factor is PPT, the socket power allowance. TDC limits sustained current and EDC peak current [2]. Start with 65, 105 and 150 W plus stock/default: four profiles in total. The 9950X's 170 W TDP is not its stock PPT [1, 2]. We will save the current BIOS profile, then use stock BIOS CPU settings for the baseline, without manual CPU changes or motherboard enhancements. We will screenshot all power, current and temperature limits, check what Auto applies, and save the BIOS profile and screenshots with the test files. Memory settings stay the same in every test.

In the manual profiles, keep stock baseline TDC/EDC, Curve Optimizer at zero, boost override at zero, default scalar and thermal protection enabled. SMT stays on; core count, memory profile/timings, GPU settings, fan curve and Windows plan stay fixed. If entering PPT requires manual PBO, document that and explicitly match the other controls [2]. Do not use a motherboard 'unlimited' preset. If a proposed limit exceeds the verified stock ceiling, revise the set to stay at or below stock before the main campaign; remove duplicate limits.

Pilot and run order

We will try each PPT setting first, check for errors, and confirm the limit in HWiNFO. Check the memory, outputs and timestamps too. Compare monitoring on/off in three paired pilot runs of Blender and each 3DMark benchmark. If the difference exceeds pilot variation, simplify logging and freeze one interval. Confirm 3DMark test boundaries and GPU use; a graphics test need not be CPU-limited. Raise Blender's sample count once if the stock render is shorter than three minutes, then save that scene.

Use 12 blocks, each with a saved random order of the four power profiles. Reboot after each profile change and verify it in HWiNFO. Run Blender, 7-Zip, HandBrake, Time Spy and CPU Profile once per profile, rotating their order between blocks. This gives 5 workloads x 4 profiles x 12 repetitions = 240 benchmark runs. Sub-tests within a run are not extra repetitions. Investigate large pilot variation before collection; keep all valid runs, including slow ones.

At each profile, give each workload one unrecorded warm-up. Before every measured run, wait at least two minutes and require CPU idle temperature within 2 C of the session's idle reference for 60 seconds. Record room temperature with the same thermometer; pause if it drifts more than 2 C from the session start. If the PC has not settled after ten minutes, check why. Close background applications and warm the input cache consistently rather than mix cold and warm starts.

Protocol for each workload

Blender	Run the saved CPU-only scene in background mode. Capture launch/exit times, render log and output PNG. Accept only a successful render with the expected dimensions and sample count.
7-Zip	Run the fixed command from Section 3. Save complete stdout, launch/exit times and both average ratings. Treat its ten internal iterations as one trial; reject a reported data error, not a low rating.
HandBrake	Load the frozen input and preset; run one CPU encode to a unique output file. Save the activity log, launch/exit times and output duration/frame count. Verify the same 180-second segment, dimensions and encoder each time.
3DMark	Run each benchmark with its frozen settings. Save the result file, individual scores, settings and sensor CSV. Capture graphics frame times where supported. Use the pilot's test boundaries; exclude demos and loading from energy. Keep Time Spy CPU/graphics and CPU Profile thread-count results separate.

We will log HWiNFO for 30 seconds before and after each run. This is our buffer for samples around the task boundaries and a view of the return towards idle, not a published timing rule. Energy covers only the task interval. For process jobs, record launch/exit timestamps on the CSV clock; for 3DMark, use verified test boundaries and graphics timestamps. Align boundaries within one logging interval and record that uncertainty. Flag gaps above twice the fixed interval, missing power values, crashes or background interruptions. Save failed runs with a reason and repeat them. A large value alone is not a reason to delete it.

7. Timeline for the rest of the semester

The dates below are a plan from 1 October onward, not a record of finished work. They continue the Task 1 schedule; the final conference date still needs confirmation.

1-11 October	Apply design feedback, confirm 3DMark access and prepare the inputs.
12-18 October	Finish the CPU and cooling changes early in the week, record the setup, run pilots and freeze settings, versions and inputs.
19 Oct-1 Nov	Collect the 12 randomised blocks. Check logs and outputs after each session and back them up.
2-8 November	Review failures, repeat documented invalid runs and finish the planned repetitions.
9-15 November	Use the logs to calculate task time, performance relative to stock, package energy and efficiency. Summarise temperature, variation and confidence intervals, then plot and compare the workloads.
16-22 November	Revise the report, check claims against raw data and prepare the presentation.
23 Nov, tentative	Project conference, following the provisional date used in Task 1.

8. Discussion and data management

Keep one folder per run, such as block01_ppt105_blender, containing raw CSVs, tool output and a short run record. Record profile, order, start/end times, ambient temperature, input hash, versions and any interruption. Keep configurations and pilot data separately. Raw files stay unchanged; derived tables must identify their source runs. Back up the folder after every session, with both participants able to access it. Share presets and permitted inputs, not proprietary benchmark assets.

The main risks are sensor error, background activity and short tests with too few power samples. Fixed logging helps comparison but does not calibrate the sensor. The current 8000 MT/s profile is stable; check it again after the CPU swap. Keep temperature protection on and stop tests that crash or report errors.

GPU-focused tests may barely react to PPT changes, unlike CPU-heavy tests. Neither predicts every game. CPU Profile is brief, so sustained rendering and encoding still matter [15]. Before the pilot, record the final cooling parts, case and PSU models, and save fan and pump settings. Keep these fixed; conclusions apply to this PC and these tests, including our single scene and video.

References

Documentation accessed 1 October 2026; CPU, statistics and 3DMark references checked 9 October. References support technical definitions. Run count, logging buffers and pilot thresholds are our design choices, to be checked in the pilot.

[1] AMD. Ryzen 9 9950X specifications. Processor specifications.

[2] AMD. Ryzen Master User Guide, CPU controls. PPT, TDC and EDC controls.

[3] HWiNFO. Software capabilities and sensor logging. Monitoring and logging.

[4] Blender Foundation. Blender 4.5 LTS Manual, command-line arguments. CPU rendering options.

[5] Blender Foundation. Demo Files: Classroom, by Christophe Seux, CC0. Scene source.

[6] Igor Pavlov. 7-Zip 25.01 User Manual, b (Benchmark) command. Bundled 7-zip.chm. Official distribution.

[7] HandBrake. Command line reference. Encode options.

[8] HandBrake. Performance documentation, including Tears of Steel test source. CPU encoding and source video.

[9] Intel / GameTechDev. PresentMon Console documentation. Frame CSV definitions.

[10] UL Solutions. 3DMark Time Spy technical documentation. Tests and scoring.

[11] G. Raffin and D. Trystram. Dissecting the Software-Based Measurement of CPU Energy Consumption: A Comparative Analysis. IEEE TPDS 36(1), 96-107, 2025. Research paper.

[12] NIST/SEMATECH. e-Handbook of Statistical Methods, confidence limits for the mean. Confidence interval definition.

[13] NIST. Dataplot Reference Manual, confidence limits for the mean. Normality assumption.

[14] NIST/SEMATECH. e-Handbook of Statistical Methods, underlying assumptions. Independent, stable measurements.

[15] UL Solutions. CPU Profile tests. Thread counts and workload.